Assignment 1 Report

Deep Learning for Space, Time and Graphs – Fall 2026

Muhammad Ibrahim Anwar
27100154

October 5, 2026

Repository

GitHub repository: https://github.com/ibrahim234523/DLSTG_PA1

Task 1

1 Response 1A: Predict, Measure, and Revise the Aggregate Ordering

Pre-registered prediction (before running Output 1.1)

- **Established stations, expected ranking (worst → best):** Shared ridge → Sensor-specific ridge → Raw Attention → Period-routed ridge.
- **Station 4, expected ranking (worst → best):** Sensor-specific ridge → Shared ridge → Period-routed ridge → Raw Attention.
- **Reasoning:** Period routing was expected to perform better on known stations since the periodic features of the signals would perform better since it aims to learn exact period related forecast. Raw Attention was thought to to win on Station 4 since it was held out, so its data was not seen in training, so zero-shot forecast was expected to perform better relatively. Sensor-specific ridge was expected to fail out-of-distribution on unseen station since there was no learning for the held out station 4.

Measured results (Output 1.1)

Table 1: Validation RMSE / MSE of the four Part-1 forecasters, established stations vs. Station 4.

Forecaster	RMSE (Est.)	RMSE (Station 4)	MSE (Est.)	MSE (Station 4)
Shared ridge	0.767	0.876	0.588	0.767
Sensor-specific ridge	0.768	*unavailable*	0.590	*unavailable*
Period-routed ridge	**0.166**	**0.173**	**0.028**	**0.030**
Raw Attention	0.420	0.615	0.176	0.378

Figure 1: Output 1.1: forecaster comparison across Products A, B, C.

Revised ordering and explanation

Revised Measured Ranking (worst → best):

- **Established stations:** Sensor-specific ridge (0.768) ≈ Shared ridge (0.767) → Raw Attention (0.420) → Period-routed ridge (0.166).
- **Station 4:** Sensor-specific ridge (*unavailable*) → Shared ridge (0.876) → Raw Attention (0.615) → Period-routed ridge (0.173).

Comparison over Stations and Products:

- **Station Comparison:** Period-routed ridge decisively wins across both populations, degrading by only +0.007 m/s^2 (0.166 → 0.173). In contrast, Raw Attention degrades sharply by +46% (0.420 → 0.615), and Sensor-specific ridge is completely unavailable for Station 4.
- **Product Comparison:** Across all three products (Product A: $T \approx 16.1$, Product B: $T \approx 24.0$, Product C: $T \approx 30.1$), Period-routed ridge achieves consistently low RMSE (0.150–0.184 m/s^2). Meanwhile, Shared ridge (0.621–0.982) and Raw Attention (0.386–0.653) fluctuate widely depending on cycle speed with no clear trend.

Why Models Diverge and What Assumptions are present:

1. **Period Feature is the true physical driver:** Vibration depends primarily on the product speed (T), not where the sensor is situated. Routing by period isolates and learns different speeds and different operating paces, avoiding confusion for forecast.
2. **Station identity provides nor helpful information:** Training separate models per station gives zero improvement on known stations (0.768 vs. 0.767) as according to the table above. In addition, this does not apply for station 4 sinc eit is unseen.
3. **Attention-wise learning is complex:** With 368k parameters, Raw Attention tends to overfit to local noise. This shows that treating time steps as flexible token embeddings is unnecessary for simple harmonic signals

...

2 Response 1B: Why the Forecasting Rule Must Depend on Context

The fixed-matrix compromise

A single linear matrix $\hat{y} = x\hat{W}$ finds it difficult to align and predict two different frequencies at once. When Station 1 processes both Product A ($T \approx 16.1$ samples) and Product C ($T \approx 30.1$ samples), a static $\hat{W}$ compromises by dampening wave amplitudes and mistiming crests. This compromise shows us how the performance degrades : Shared ridge achieves an RMSE of only 0.779 m/s^2 on Product A and 0.448 m/s^2 on Product C. Sensor-specific ridge suffers the same fate (0.787 and 0.436 m/s^2) because station identity is not useful in separating and identifying periodic trends.

Context-dependent mixing in Attention

Attention computes its mixing weights in a dynamic sense using the context: $A(x) = \text{softmax}(QK^T/\sqrt{d})$. Despite sharing identical, frozen projection matrices (W_Q, W_K, W_V), Output 1.3 reveals a **maximum weight difference of 0.521** (mean difference 0.011) between products. As shown in Figure 2, Product A focuses heavily on a vertical band at **key position ≈ 69**, while Product C shifts its attention to recent steps (**keys 85–95**). The difference map clearly displays this divergence through the orange stripe at key 69 ($A > C$) and blue patches near the boundary ($C > A$). This demonstrates that Attention dynamically calculates and mixes the weights despite using static key,value and query matrices, the context over here is the main driving force.

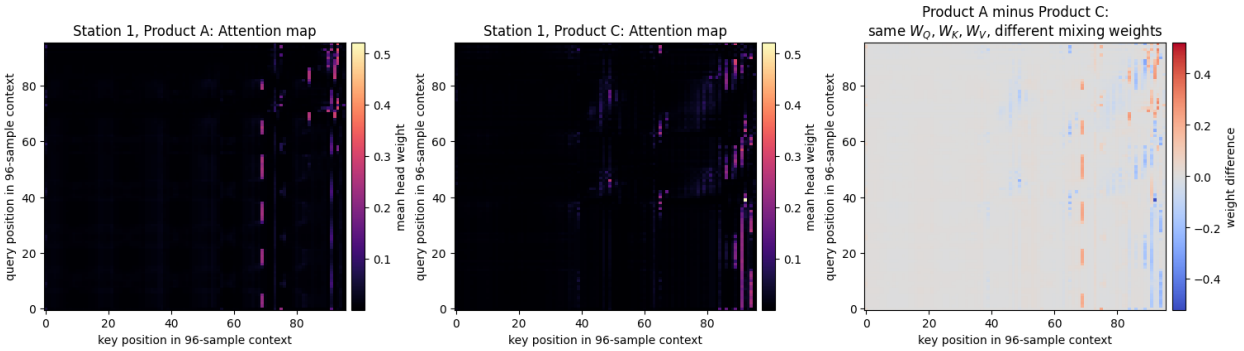


Figure 2: Output 1.3: Attention maps for Product A vs. Product C (6,000-sample run) showing up to a 0.521 weight difference under identical projection matrices.

...

3 Response 2: Series Decomposition

Selected width and observed changes (Outputs 2.1–2.3)

We select kernel width $m = 41$. Product C operates at cycle length $T \approx 30.1$–32.0 samples. The averaging window must strictly exceed the period ($m > T$) so that positive and negative phases cancel out within the moving average.

- In Figure 3, width 17 is a small minute window and it also absorbs the wave amplitudes instead of catching the full periodic harmonic peak, squishing the vibration down to 0.58 std. At width 41, the trend becomes a much smoother slope, letting the vibration keep its full natural height of 1.57 std and cleanly revealing the 32-sample cycle in the third plot.
- In Figure 4, the uphill slope on raw data confuses the forecaster as the period exceeds the window ($m < T$), resulting in finding the true cycle only 68.7 percent of the time. Removing the slope with width 41 makes the rhythm concrete and identifiable, boosting detection accuracy to 99.8 percent.
- In Output 2.3 (Table 2), splitting the signal improves the forecast accuracy across all three products, working consistently on both the familiar stations and the unseen Station 4:
  - **Established stations:** Mean RMSE drops from 0.420 → **0.297** m/s^2 (MSE cut in half from 0.176 → **0.088**). Every product improves: Product A (0.386 → 0.269), Product B (0.458 → 0.339), and Product C (0.412 → 0.279).

– **Held-out Station 4:** Mean RMSE drops from 0.615 → **0.445** m/s^2 (MSE 0.378 → **0.198**). Product A achieves the largest gain, plunging from 0.653 → **0.376** m/s^2 (a 42% error reduction).

Table 2: Validation comparison: Raw Attention vs. Attention + Decomposition ($m = 41$).

Model	RMSE (Est.)	MSE (Est.)	RMSE (Station 4)	MSE (Station 4)
Raw Attention	0.420	0.176	0.615	0.378
Attention + Decomposition ($m = 41$)	**0.297**	**0.088**	**0.445**	**0.198**

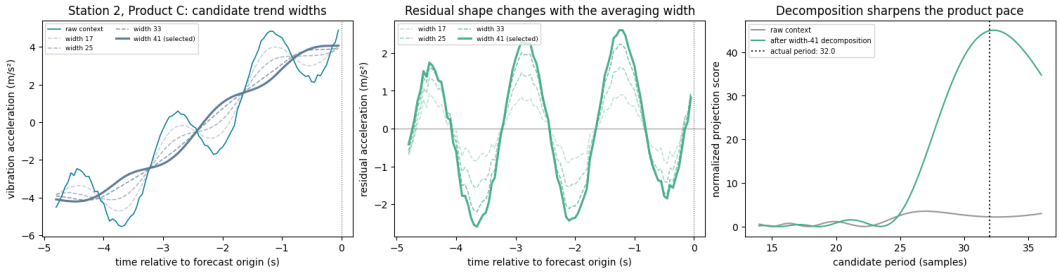


Figure 3: Trend and residual shapes across candidate widths (17, 25, 33, 41) and period sharpening for Product C.

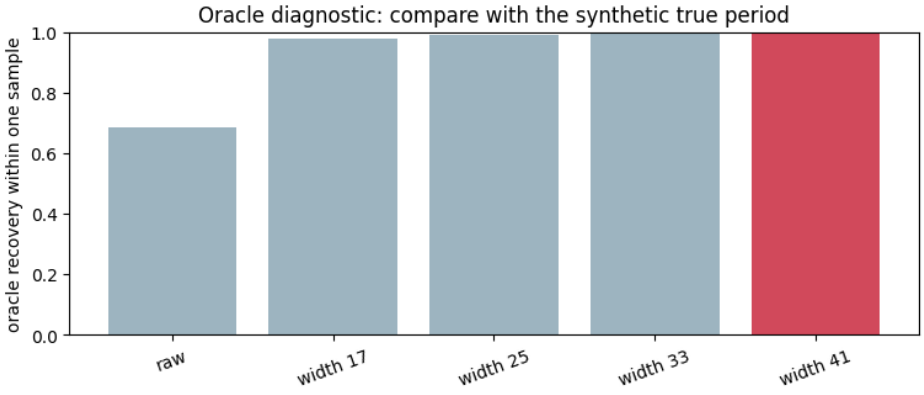


Figure 4: Oracle period recovery within one sample. Removing the slow drift with width 41 (red) raises period detection accuracy from 68.7% on raw data to 99.8%.

Why a moving average is a sensible inductive bias

A moving average works because the two different processes (time and amplitude) are operate at completely different speeds:

- Slow background drift changes over ~ 240 samples (time).
- Fast machine vibration repeats every 14 to 36 samples (amplitude).

A window of $m = 41$ sits right in the middle ensuring separate learning for both of the processes. It is wide enough to capture one full vibration cycle ($41 > 36$) without lagging behind the slow drift. The oracle diagnostic is completely safe and non-leaking because the true period is never fed directly, and it is is used only offline on the training set to help select the best time window $m = 41$. It is never fed into the neural network and never used at test time.

A failure case or alternative decomposition

This moving average fails during a trends with abrupt changes or sharp turns, such as an instant machine shutdown or spiky real world data. Because it assumes all drift is gradual, it portrays a sudden vertical jump into an inconsistent ramp projection, which compromises the learning for both the trend and the residual. An alternative is using a Median Filter. It assumes the baseline has sharp jumps between flat steps and uses the median value output to minimize the error. It preserves sudden edges cleanly, but computing median becomes costly and the smooth differentiability and gradient computation is also affected.

...

4 Response 3: Mix by Temporal Delay

Recurrence diagnostic (Output 3.2)

In Output 3.2, the residual signal exhibits strong recurrence at integer lags of **16** and **32** samples:

- Lag 16 achieves a correlation of **0.896**, corresponding to the primary period ($1P \approx 16.135$ samples).
- Lag 32 achieves a correlation of **0.897**, corresponding to the second harmonic multiple ($2P \approx 32.269$ samples).
- A third recurring peak is visible at lag 48 ($3P \approx 48.4$ samples).

These lags line up directly with the operating run's true physical period P and its integer multiples $(1P, 2P, 3P)$.

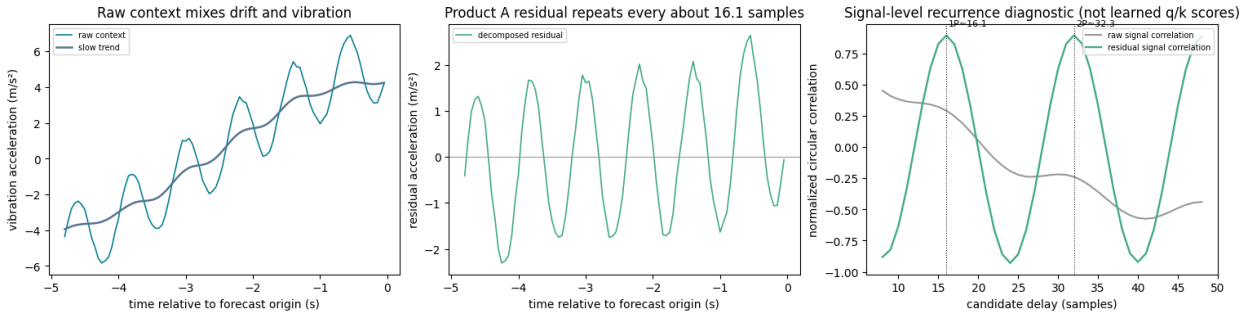


Figure 5: Output 3.2: Recurrence diagnostic for Product A showing strong periodic correlation at lags 16 and 32.

Why decomposition makes delay evidence cleaner

Removing the slow background drift is critical because a monotonic slope ruins the periodic circular correlation. When two segments on an upward ramp are shifted and compared the offsets often don't align properly causing and capping raw correlation below 0.45. Once the trend is subtracted, the residual oscillates symmetrically around zero with constant amplitude. Shifting this zero-mean wave by 16 or 32 samples aligns the peaks and troughs cleanly, causing correlation to jump to ≈ 0.90. This validates the delay mixer's inductive assumption that fixed time windows or alignment segments perform better rather than computing an expensive full attention matrix across all 96×96 token pairs.

A failure case for delay-based mixing

Delay-based mixing fails when a machine is rapidly accelerating or braking within a single window (a drifting period), or when an isolated, one-time shock occurs.

- If the machine speed changes continuously inside the window (for example, the cycle shrinks from 32 samples down to 16 samples), no single fixed shift τ can line up the early wide waves with the late narrow waves. The global correlation blurs out and misses the pattern.
- If a one-time random bump occurs, shifting the entire sequence by τ copies that single spike forward in time, creating false "ghost echoes" in the forecast for an event that will not actually repeat.

5 Response 4: Compare Designs and Deploy

(a) Pre-registered prediction

Decomposition should be the technique that will provide the most help most when slow drift is prominent because it delegates and separates the slow moving part and the jumping vibrations,allowing the token mixer in representing both differently. Delay mixing helps in primarily aligning the recurring oscillations and struggles when strong baseline drift distorts circular alignment.

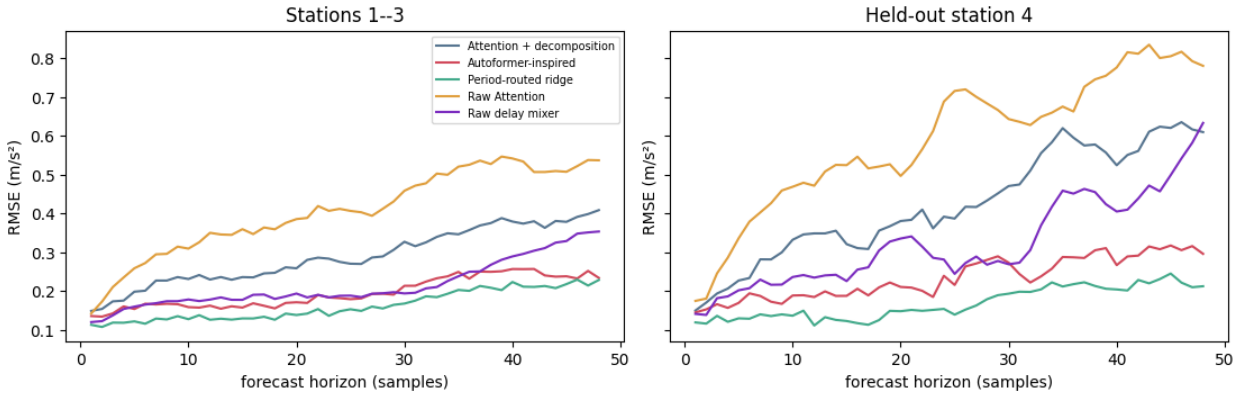
(b) Validation results across the four neural models and linear baselines

Table 3: Validation RMSE / MSE, established stations: four neural variants and linear baselines.

Model	RMSE (m/s^2)	MSE (m/s^2)^2
Shared ridge	0.767	0.588
Period-routed ridge	**0.166**	**0.028**
Raw Attention	0.420	0.176
Attention + Decomposition	0.297	0.088
Raw Delay Mixer	0.223	0.050
Autoformer-inspired (Decomp. + Delay Mixer)	0.199	0.040

Isolated vs. Combined Gains: Starting from Raw Attention (MSE = 0.176), decomposition alone cuts MSE by 0.088 (to 0.088), and delay mixing alone cuts MSE by 0.126 (to 0.050). This combined gain is slightly less than adding the two separate improvements together ($0.136 < 0.088 + 0.126$), meaning the two upgrades share some overlapping benefits. However, on the unseen Station 4, combining them works remarkably well, slashing MSE from 0.378 down to 0.058 (an 85% drop).

Horizon Error Growth (Output 4.2): As horizon $h \to 48$, Raw Attention's error increases sharply (climbing to 0.54 m/s^2 on established stations and > 0.80 m/s^2 on Station 4). In contrast, both delay-based architectures stay flat and stable below 0.32 m/s^2. Period-routed ridge remains the flattest overall, maintaining near-constant error (≈ 0.12–0.22 m/s^2) across all 48 steps.



5.1 (c) Deployment model selection

Established Stations: Period-routed ridge. It achieves the lowest validation error (RMSE = 0.166 m/s^2, MSE = 0.028), beating the best neural model, which is the Autoformer-inspired (0.199 m/s^2). It uses fewer parameters (59,904 vs. 373,026) and training time is also instantaneous (0.003 s vs. 688.7 s). Routing by operating period provides the exact inductive bias needed for stationary harmonics signals.

Station 4): Period-routed ridge. It provides the strongest generalization (RMSE = 0.173 m/s^2, test RMSE = 0.183 m/s^2), degrading by only +0.007 m/s^2 from established stations. This is primarily as the product harmonic signal is primary driver for the forecast in this case, resulting in frequency routing transfers zero-shot, easily surpassing Autoformer-inspired.

...

6 Part 2: Real-World Forecasting with Autoformer

Data Processing and Period Detection

The raw target is heavily skewed with extreme spikes, so we applied a $\log(1+x)$ transform and z-score normalization to create a stable bell curve (Figure 6). Both FFT and autocorrelation confirmed an exact 24-hour cycle (with ACF harmonic peaks at 384 and 720 lags), leading us to align input windows to weekly cycles (168/336 steps) and set the decomposition kernel to $m = 25$.

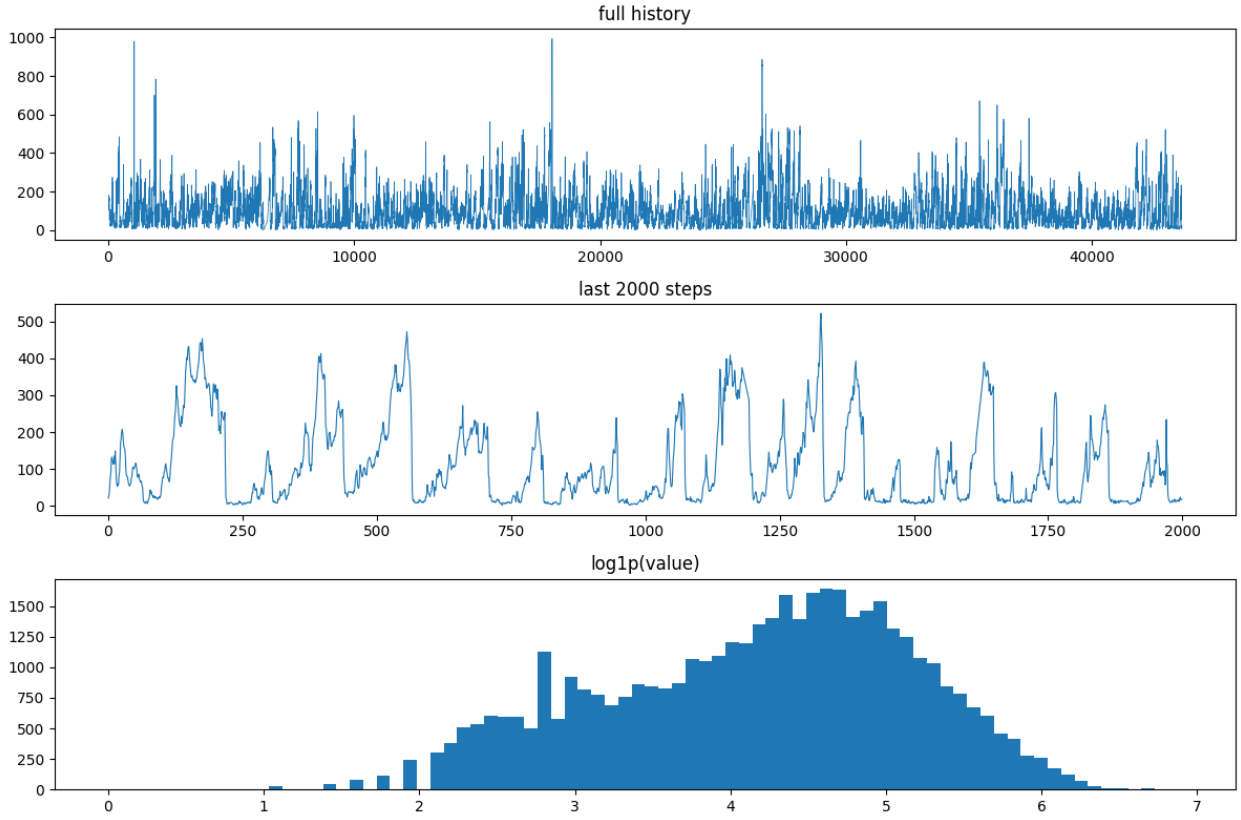


Figure 6: Target signal exploration: raw 44,000-step history (top), zoomed 24-hour cycles over 2,000 steps (middle), and the normalized bell curve after $\log(1+x)$ transformation (bottom).

Covariate Analysis and Ablation

The external features contain continuous sensors and four mutually exclusive operating states (G–J). Feature D ($\rho = -0.245$) and State H ($\rho = -0.212$) trigger significant drops, while Feature A and State J signal demand surges.

Covariate Ablation: With vs. Without External Data

Without external data (`none`), the model collapses to a flat historical mean with high error (109.53 $\mathrm{m/s^2}$). Supplying future co-variates (`future`) resolves this, dropping validation RMSE to 66.25 $\mathbf{m/s^2}$ (best seed: 67.79 $\mathbf{m/s^2}$)

Table 4: Lagged correlation of external covariates with the target across offsets k (hours). Offsets $k < 0$ represent past values; $k > 0$ represent future values.

Covariate	$k = -168$	$k = -24$	$k = -1$	$k = 0$	$k = +1$	$k = +24$	$k = +168$
Continuous Variables							
Feature A	-0.073	0.024	0.167	0.169	0.168	0.058	-0.082
Feature B	-0.116	-0.106	-0.088	-0.088	-0.086	-0.069	-0.109
Feature C	0.091	0.089	-0.047	-0.048	-0.050	0.024	0.083
Feature D	-0.008	-0.148	**-0.245**	**-0.245**	**-0.243**	-0.074	-0.017
Feature E	0.036	-0.006	0.021	0.019	0.019	0.035	0.011
Feature F	-0.027	-0.059	-0.055	-0.050	-0.044	0.044	-0.020
Binary Variables							
Feature G	-0.002	-0.019	-0.032	-0.033	-0.031	0.021	0.003
Feature H	0.023	-0.171	**-0.243**	**-0.212**	-0.182	-0.010	0.015
Feature I	-0.025	0.086	0.115	0.097	0.079	-0.022	-0.028
Feature J	0.005	0.110	0.168	0.154	0.139	0.021	0.013

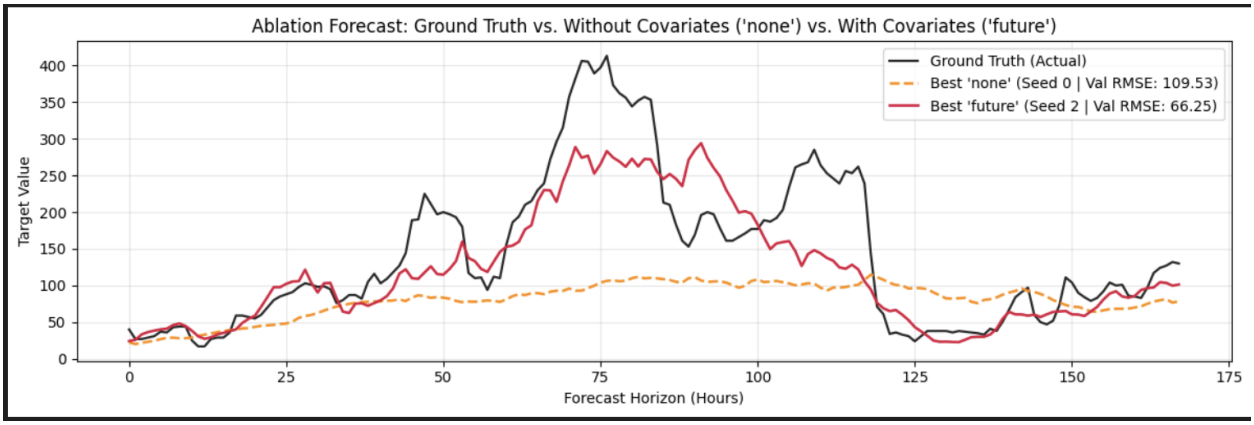


Figure 7: Validation comparison: ground truth (black) vs. flat forecast without covariates (`none`) vs. dynamic tracking with future covariates (`future`).

Deployment and Leaderboard Reflection

For final deployment, we generated an ensemble forecast with future covariates (, $P = 121{,}025$, $E = 51$), averaging predictions across top-performing seeds to reduce single-model variance. Figure 8 displays the resulting 168-hour ensemble forecast.

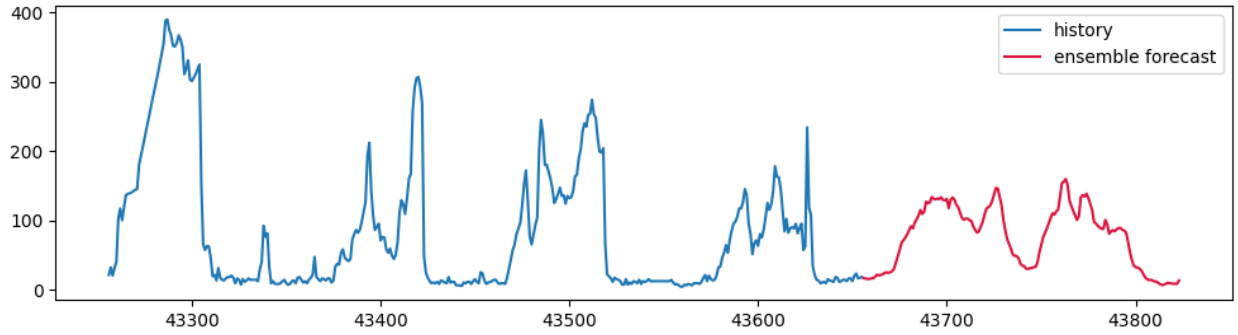


Figure 8: Final 168-hour ensemble forecast over the hidden test horizon (crimson) following recent history (blue).

Official Leaderboard Results and Insight: On the official leaderboard, our submission achieved a test RMSE of 104.46 $\mathbf{m/s^2}$ (MAE 71.73 m/s^2, sMAPE 63.55%, and overall leaderboard score 104.4753).

While our models averaged ≈ 73.26 m/s^2 across 10 validation weeks, the single test horizon highlights the classic trade-off of squared-error metrics: the ensemble acts as a smooth, risk-balanced predictor ($\approx$ 130–160 peak heights) that avoids catastrophic over-prediction penalties, but incurs quadratic error if that specific hidden test week experiences an unpredicted extreme surge.